NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

Department of Computer Science and Engineering

LABORATORY MANUAL

Java and Spring Framework Laboratory

CS3603-1

Academic Year 2026–27

NAME Prajwal Jogi

USN

SEMESTER / SECTION

Index

Java and Spring Framework Laboratory · CS3603-1 · Prajwal Jogi

EXPT. NO.	DATE	TITLE OF THE EXPERIMENT	PAGE NO.	SIGNATURE
Setup		Lab environment setup (Java, Tomcat 9, Eclipse, MySQL)	iii	
1a		Servlet handling HTTP GET and POST requests	6	
1b		Session tracking using cookies and session objects	8	
2a		JSP application for employee information	11	
2b		Server-side validation using JSP	14	
3		JSP and JDBC: CRUD operations on the students table	18	
App. A		Practical execution and compatibility guide	27	

Lab Environment Setup

The experiments are carried out on a Linux (Ubuntu) machine with the software listed below. This section gives the installation steps. A click-by-click guide for installing VirtualBox and the Linux image on Windows 10/11 or macOS is available at https://lab-guide-ten.vercel.app/.

SOFTWARE	VERSION / NOTE
Operating system	Ubuntu Linux (lab PC, or an Ubuntu virtual machine in VirtualBox)
Java	OpenJDK 11 or later
Web server	Apache Tomcat 9.0.x
IDE	Eclipse IDE for Enterprise Java and Web Developers
Database	MySQL Server 8.0
JDBC driver	MySQL Connector/J 8.x

1 Get a Linux machine

On the lab PC, Linux is already installed; go to Step 2. On your own computer, install Oracle VirtualBox, create a 64-bit Ubuntu virtual machine (at least 4 GB RAM and 25 GB disk), install or import the Linux image and start the machine. Follow the guide at https://lab-guide-ten.vercel.app/ for the exact clicks on Windows and macOS.

2 Install Java (JDK)

Open a terminal and run:

TERMINAL

```
$ sudo apt update
$ sudo apt install openjdk-17-jdk
$ java -version
$ javac -version
```

3 Install Apache Tomcat 9

Download the binary distribution (tar.gz) of Tomcat 9 from https://tomcat.apache.org/download-90.cgi, then extract it into your home folder as `tomcat`:

TERMINAL

```
$ tar -xzf apache-tomcat-9.0.*.tar.gz
$ mv apache-tomcat-9.0.* ~/tomcat
$ chmod +x ~/tomcat/bin/*.sh
$ ~/tomcat/bin/version.sh
$ ~/tomcat/bin/startup.sh
```

Open http://localhost:8080 in the browser; the Tomcat home page should appear. Stop the server with `~/tomcat/bin/shutdown.sh`.

4 Install Eclipse

Download *Eclipse IDE for Enterprise Java and Web Developers* from https://www.eclipse.org/downloads/packages/, extract the archive, start the `eclipse` program and choose a workspace folder.

5 Add Tomcat to Eclipse

1. Open **Window > Preferences > Server > Runtime Environments** and click **Add...**
2. Select **Apache Tomcat v9.0** and click **Next**.
3. Set **Tomcat installation directory** to the folder where Tomcat was extracted, choose the installed JRE and click **Finish**.
4. Open **Window > Show View > Servers**, right-click in the view and choose **New > Server > Apache > Tomcat v9.0 Server**, then click **Finish**.

6 Install MySQL and the JDBC driver

TERMINAL

```
$ sudo apt install mysql-server
$ sudo systemctl start mysql
$ sudo mysql
mysql> ALTER USER 'root'@'localhost'
    ->   IDENTIFIED WITH mysql_native_password BY 'your_password';
mysql> EXIT;
```

Log in later with `mysql -u root -p`. Download *MySQL Connector/J* (Platform Independent ZIP) from https://dev.mysql.com/downloads/connector/j/ and extract the JAR; a web project that connects to MySQL needs this JAR in its `WEB-INF/lib` folder.

7 Create and run a web project

1. Choose **File > New > Dynamic Web Project**, enter a project name, set **Target runtime** to **Apache Tomcat v9.0** and **Dynamic web module version** to **4.0**, tick **Generate web.xml deployment descriptor** and click **Finish**.
2. Put Java classes under **Java Resources > src** (create the package first) and JSP/HTML files under `src/main/webapp`.
3. Right-click the project and choose **Run As > Run on Server > Tomcat v9.0 > Finish**, then open the URL in the browser.

1a

EXPERIMENT NO. 1A

Servlet handling HTTP GET and POST requests

AIM

Implement a Java Servlet (SimpleServlet) that handles both HTTP GET and POST requests. The servlet should:

- For GET requests, respond with a simple HTML page that includes a form to submit a name parameter.
- For POST requests, retrieve username and password parameters from the request and respond with an HTML page displaying these parameters.

Include necessary servlet methods to read data from the client and send appropriate responses.

PROGRAM

SimpleServlet.java — JAVA

```java
package com.example;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/SimpleServlet")
public class SimpleServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h2>Submit Your Details</h2>");
        out.println("<form method='POST' action='SimpleServlet'>");
        out.println("Name: <input type='text' name='name'><br><br>");
        out.println("Password: <input type='password' name='password'><br><br>");
        out.println("<input type='submit' value='Submit'>");
        out.println("</form>");
        out.println("</body></html>");
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String name = request.getParameter("name");
        String password = request.getParameter("password");
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
```

```
38          out.println("<html><body>");
39          out.println("<h2>Submitted Data</h2>");
40          out.println("Name: " + name + "<br>");
41          out.println("Password: " + password + "<br>");
42          out.println("</body></html>");
43      }
44  }
```

OUTPUT



GET request: form returned by doGet()



POST request: data returned by doPost()

1b

EXPERIMENT NO. 1B

Session tracking using cookies and session objects

AIM

Implement a Java Servlet (SessionServlet) to demonstrate session tracking using cookies and session objects. The servlet should:

- Display a login form when accessed initially.
- Handle login functionality via a POST request, validating username and password.
- Upon successful login, store username in a session attribute and display a welcome message with logout option.
- Implement logout functionality to invalidate the session upon user request.
- Display session information including session ID, creation time, and last accessed time on the response page.

PROGRAM

SessionServlet.java — JAVA

```java
package com.example;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet("/SessionServlet")
public class SessionServlet extends HttpServlet {

    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            /* TODO output your page here. You may use following sample code. */
            out.println("<!DOCTYPE html>");
            out.println("<html>");
            out.println("<head>");
            out.println("<title>Servlet SessionServlet</title>");
            out.println("</head>");
            out.println("<body>");
            out.println("<h1>Servlet SessionServlet at " + request.getContextPath() + "</h1>");
            out.println("</body>");
            out.println("</html>");
        }
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
```

```
            throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Check if the logout action is requested
        String action = request.getParameter("action");
        if ("logout".equals(action)) {
            HttpSession session = request.getSession(false);
            if (session != null) {
                session.invalidate(); // Invalidate the session
            }
            response.sendRedirect("SessionServlet"); // Redirect to the login page
            return;
        }

        // Check if session exists
        HttpSession session = request.getSession(false);
        if (session != null && session.getAttribute("username") != null) {
            // User is logged in, display welcome message and session info
            out.println("<html><head><title>Welcome</title></head><body>");
            out.println("<h2>Welcome, " + session.getAttribute("username") + "!</h2>");
            out.println("<p><a href='SessionServlet?action=logout'>Logout</a></p>");
            out.println("<h3>Session Information:</h3>");
            out.println("<p>Session ID: " + session.getId() + "</p>");
            out.println("<p>Creation Time: "
                    + new java.util.Date(session.getCreationTime()) + "</p>");
            out.println("<p>Last Accessed Time: "
                    + new java.util.Date(session.getLastAccessedTime()) + "</p>");
            out.println("</body></html>");
        } else {
            // User is not logged in, display login form
            out.println("<html><head><title>Login</title></head><body>");
            out.println("<h2>Login</h2>");
            out.println("<form method='post' action='SessionServlet'>");
            out.println("Username: <input type='text' name='username'><br><br>");
            out.println("Password: <input type='password' name='password'><br><br>");
            out.println("<input type='submit' value='Login'>");
            out.println("</form>");
            out.println("</body></html>");
        }
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        // Process login form submission
        String username = request.getParameter("username");
        String password = request.getParameter("password");

        // For demonstration purposes, validate against hardcoded credentials
        if (username != null && username.equals("admin")
                && password != null && password.equals("password")) {
            // Login successful, create a new session if none exists and store username
            HttpSession session = request.getSession(true);
            session.setAttribute("username", username);
            // Redirect to doGet to display welcome message and session info
            response.sendRedirect("SessionServlet");
        } else {
            // Login failed, redirect back to login page
            response.sendRedirect("SessionServlet");
        }
    }

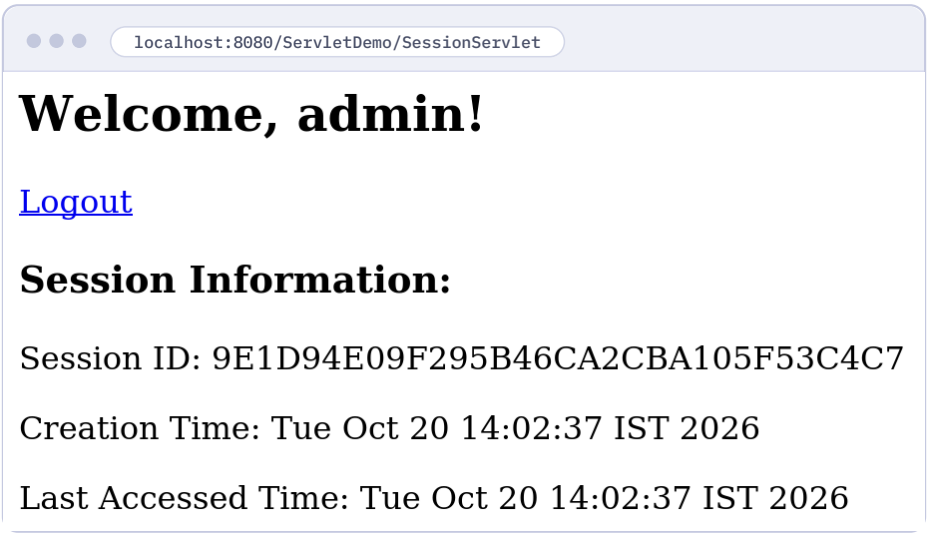
    protected void doDelete(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
```

```
98          // Process logout (via DELETE request)
99          HttpSession session = request.getSession(false);
100         if (session != null) {
101             session.invalidate(); // invalidate session
102         }
103         response.sendRedirect("SessionServlet");
104     }
105 }
```

OUTPUT



Login form (username admin, password password)



Welcome page with session information after a successful login

2a

EXPERIMENT NO. 2A

JSP application for employee information

AIM

Design a JSP application for managing employee information.

- Create a JSP page (`employeeForm.jsp`) with an HTML form to collect employee data (e.g., Employee ID, Name, Age, Department, and Email) using the POST method.
- Create a JSP page (`employeeResult.jsp`) to retrieve the submitted form data using `request.getParameter()` .
- Use JSP expressions and scriptlets to process the retrieved data. For example:
  - Concatenate the employee's name and department into a single string.
  - Calculate the birth year from the age.
  - Format the email address.
- Display the processed data in a formatted manner in `employeeResult.jsp` .

PROGRAM

employeeForm.jsp — JSP

```jsp
<!DOCTYPE html>
<html>
<head>
    <title>Employee Form</title>
</head>
<body>
    <h2>Employee Information Form</h2>
    <form action="employeeResult.jsp" method="POST">
        <table>
            <tr>
                <td>Employee ID:</td>
                <td><input type="text" name="employeeId" required></td>
            </tr>
            <tr>
                <td>Name:</td>
                <td><input type="text" name="name" required></td>
            </tr>
            <tr>
                <td>Age:</td>
                <td><input type="number" name="age" required></td>
            </tr>
            <tr>
                <td>Department:</td>
                <td><input type="text" name="department" required></td>
            </tr>
            <tr>
                <td>Email:</td>
                <td><input type="email" name="email" required></td>
            </tr>
            <tr>
                <td colspan="2" style="text-align: center;">
```

```
                    <input type="submit" value="Submit">
                </td>
            </tr>
        </table>
    </form>
</body>
</html>
```

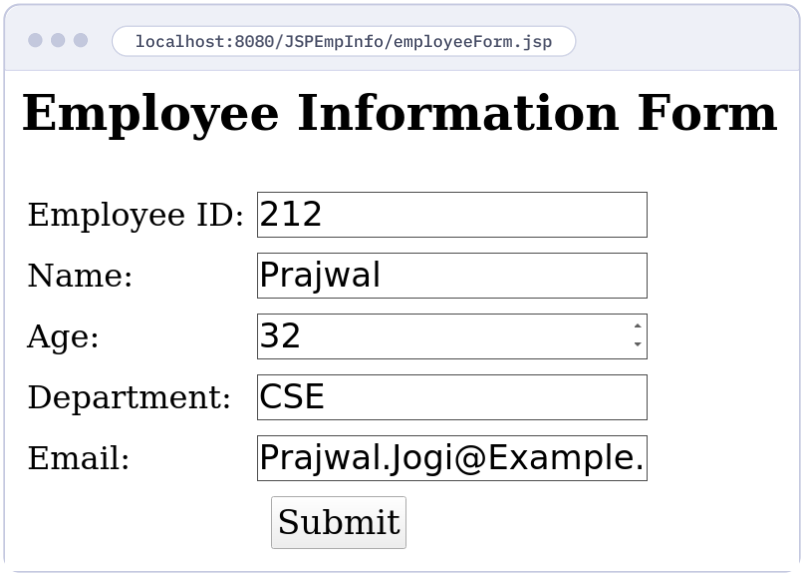
employeeResult.jsp JSP

```jsp
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ page import="java.util.Calendar" %>
<!DOCTYPE html>
<html>
<head>
    <title>Employee Information</title>
</head>
<body>
    <h2>Employee Information Result</h2>
    <%
        // Retrieve form data
        String employeeId = request.getParameter("employeeId");
        String name = request.getParameter("name");
        String ageStr = request.getParameter("age");
        String department = request.getParameter("department");
        String email = request.getParameter("email");

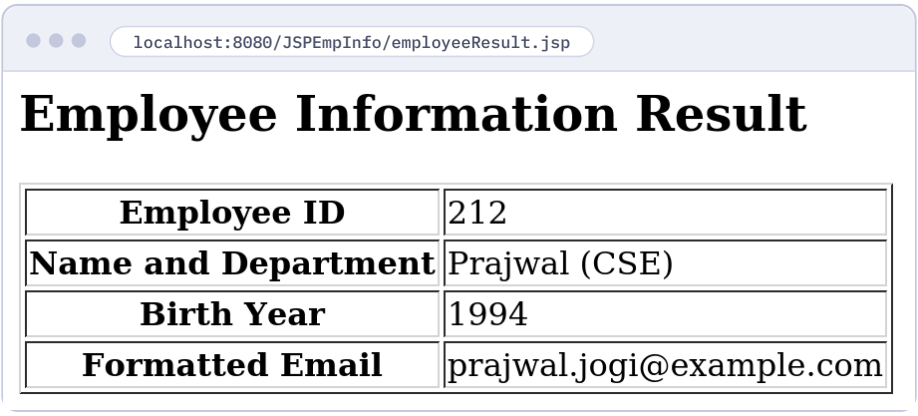
        // Process data
        int age = Integer.parseInt(ageStr);
        Calendar calendar = Calendar.getInstance();
        int currentYear = calendar.get(Calendar.YEAR);
        int birthYear = currentYear - age;
        String nameAndDepartment = name + " (" + department + ")";

        // Format email
        String formattedEmail = email.toLowerCase();
    %>
    <table border="1">
        <tr>
            <th>Employee ID</th>
            <td><%= employeeId %></td>
        </tr>
        <tr>
            <th>Name and Department</th>
            <td><%= nameAndDepartment %></td>
        </tr>
        <tr>
            <th>Birth Year</th>
            <td><%= birthYear %></td>
        </tr>
        <tr>
            <th>Formatted Email</th>
            <td><%= formattedEmail %></td>
        </tr>
    </table>
</body>
</html>
```

OUTPUT



Employee information form



Processed data displayed by employeeResult.jsp

2b

EXPERIMENT NO. 2B

Server-side validation using JSP

AIM

Enhance the `employeeForm.jsp` page to implement robust server-side validation using JSP for managing employee information.

- Ensure that all required fields (such as Employee ID, Name, Age, Department, and Email) are properly validated in `employeeResult.jsp` upon form submission.
- Implement server-side checks to verify that:
  - Employee ID is a non-empty string.
  - Name contains only letters and spaces.
  - Age is a valid numeric value between 18 and 65.
  - Department is selected from a predefined list of options.
  - Email follows a valid format (e.g., name@example.com).
- Display clear and descriptive error messages directly on `employeeResult.jsp` next to each field if validation criteria are not met, guiding users to correct their input.

PROGRAM

employeeForm.jsp JSP

```jsp
<!DOCTYPE html>
<html>
<head>
    <title>Employee Form</title>
</head>
<body>
    <h2>Employee Information Form</h2>
    <form action="employeeResult.jsp" method="POST">
        Employee ID: <input type="text" name="employeeId" required><br>
        Name: <input type="text" name="name" required><br>
        Age: <input type="number" name="age" required><br>
        Department:
        <select name="department" required>
            <option value="">Select</option>
            <option value="HR">HR</option>
            <option value="Finance">Finance</option>
            <option value="IT">IT</option>
            <option value="Sales">Sales</option>
        </select><br>
        Email: <input type="email" name="email" required><br>
        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

employeeResult.jsp JSP

```jsp
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ page import="java.util.regex.*" %>
<!DOCTYPE html>
<html>
<head>
    <title>Employee Information</title>
</head>
<body>
    <h2>Employee Information Result</h2>
    <%
        // Retrieve form data
        String employeeId = request.getParameter("employeeId");
        String name = request.getParameter("name");
        String ageStr = request.getParameter("age");
        String department = request.getParameter("department");
        String email = request.getParameter("email");

        // Initialize error messages
        String employeeIdError = "";
        String nameError = "";
        String ageError = "";
        String departmentError = "";
        String emailError = "";

        // Validation flags
        boolean isValid = true;

        // Validate Employee ID
        if (employeeId == null || employeeId.trim().isEmpty()) {
            employeeIdError = "Employee ID is required.";
            isValid = false;
        }

        // Validate Name
        if (name == null || !name.matches("^[a-zA-Z\\s]+$")) {
            nameError = "Name must contain only letters and spaces.";
            isValid = false;
        }

        // Validate Age
        int age = 0;
        try {
            age = Integer.parseInt(ageStr);
            if (age < 18 || age > 65) {
                ageError = "Age must be between 18 and 65.";
                isValid = false;
            }
        } catch (NumberFormatException e) {
            ageError = "Age must be a valid numeric value.";
            isValid = false;
        }

        // Validate Department
        if (department == null || department.trim().isEmpty()) {
            departmentError = "Department is required.";
            isValid = false;
        } else if (!department.matches("HR|Finance|IT|Sales")) {
            departmentError = "Invalid department selected.";
            isValid = false;
        }

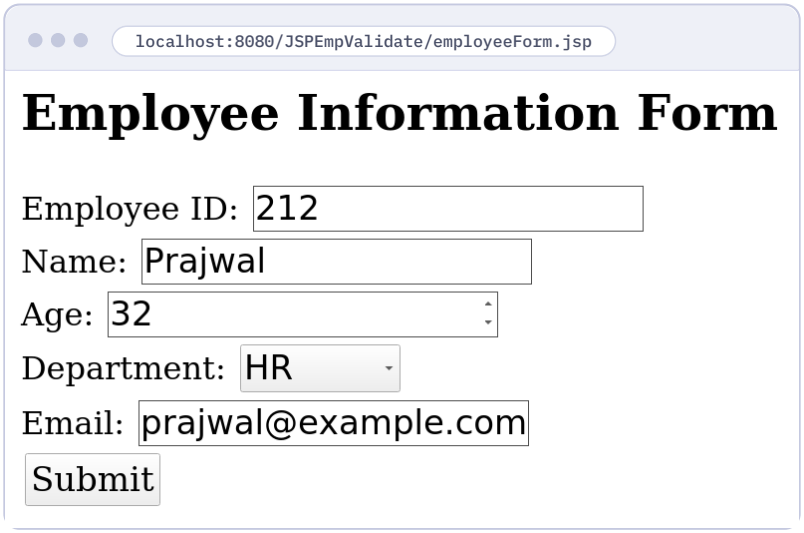
```

```
        // Validate Email
        if (email == null || !email.matches("^[\\w.-]+@[\\w.-]+\\.[a-zA-Z]{2,}$")) {
            emailError = "Email must be in a valid format (e.g., name@example.com).";
            isValid = false;
        }

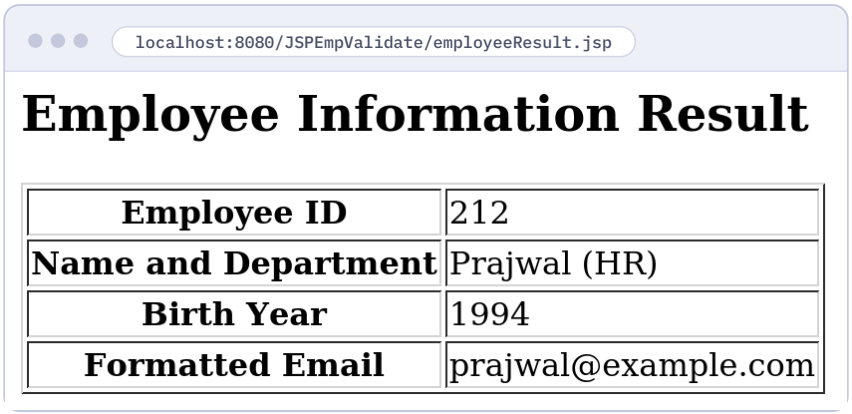
        if (isValid) {
            // Process data
            int currentYear = java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
            int birthYear = currentYear - age;
            String nameAndDepartment = name + " (" + department + ")";

            // Format email
            String formattedEmail = email.toLowerCase();
    %>
    <table border="1">
        <tr>
            <th>Employee ID</th>
            <td><%= employeeId %></td>
        </tr>
        <tr>
            <th>Name and Department</th>
            <td><%= nameAndDepartment %></td>
        </tr>
        <tr>
            <th>Birth Year</th>
            <td><%= birthYear %></td>
        </tr>
        <tr>
            <th>Formatted Email</th>
            <td><%= formattedEmail %></td>
        </tr>
    </table>
    <%
        } else {
            // Display error messages and guide the user
    %>
    <h3>Validation Errors</h3>
    <ul>
        <% if (!employeeIdError.isEmpty()) { %>
            <li><b>Employee ID:</b> <%= employeeIdError %></li>
        <% } %>
        <% if (!nameError.isEmpty()) { %>
            <li><b>Name:</b> <%= nameError %></li>
        <% } %>
        <% if (!ageError.isEmpty()) { %>
            <li><b>Age:</b> <%= ageError %></li>
        <% } %>
        <% if (!departmentError.isEmpty()) { %>
            <li><b>Department:</b> <%= departmentError %></li>
        <% } %>
        <% if (!emailError.isEmpty()) { %>
            <li><b>Email:</b> <%= emailError %></li>
        <% } %>
    </ul>
    <a href="employeeForm.jsp">Go back to the form</a>
    <%
        }
    %>
</body>
</html>
```

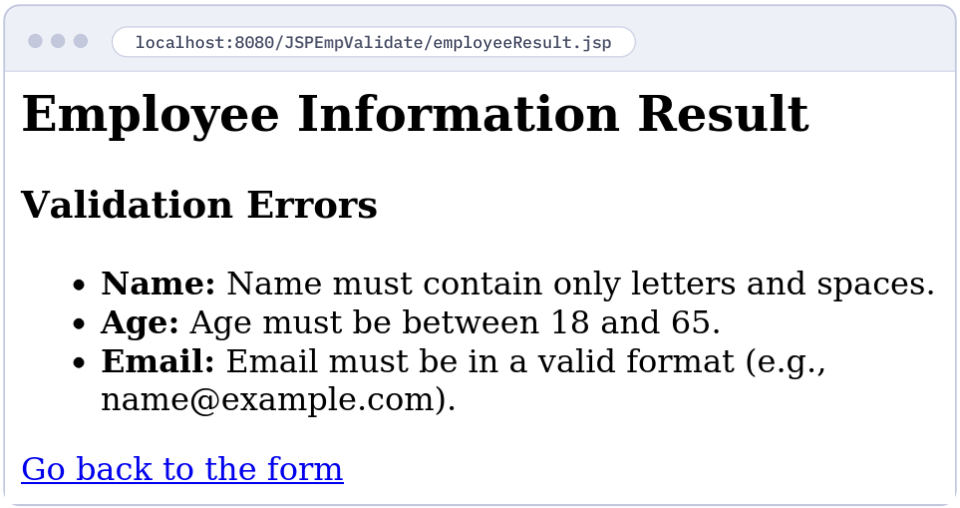
OUTPUT



Employee form with a department list



Valid input: processed data is displayed



Invalid input (Name 'Prajwal_123', Age 15, Email 'prajwal@example'): validation errors

3

EXPERIMENT NO. 3

JSP and JDBC: CRUD operations on the students table

AIM

Develop a JSP application that integrates JDBC to perform CRUD (Create, Read, Update, and Delete) operations on a `students` database table. The application should:

- **Create:** Insert new student records with fields such as roll-no, name, email, and age.
- **Read:** Retrieve and display all student records from the database.
- **Update:** Modify existing student records.
- **Delete:** Remove student records from the database.

PROCEDURE

1. Start MySQL, log in with `mysql -u root -p` (password `root123`) and run `database.sql` to create the database `studentDB` and the table `students`.
2. In Eclipse create a Dynamic Web Project named `StudentCRUD` (Target runtime: Apache Tomcat v9.0, Dynamic web module version 4.0, Generate web.xml).
3. Copy `mysql-connector-java-8.0.26.jar` into `src/main/webapp/WEB-INF/lib` (a copy is already present in Tomcat's `lib` folder on the lab PC), then refresh the project and run **Project > Clean**.
4. Create the package `com.studentcrud` with the classes `DBConnection`, `Student` and `StudentDAO`.
5. Create `addStudent.jsp`, `viewStudents.jsp`, `updateStudent.jsp` and `deleteStudent.jsp` under `src/main/webapp`.
6. Add the project to the Tomcat v9.0 server (**Add and Remove...**), start the server and open `http://localhost:8080/StudentCRUD/viewStudents.jsp`.

PROGRAM

database.sql SQL

```sql
-- Experiment 3: JSP + JDBC CRUD on the students table
CREATE DATABASE IF NOT EXISTS studentDB;
USE studentDB;

CREATE TABLE IF NOT EXISTS students (
    roll_no INT PRIMARY KEY,
    name    VARCHAR(50),
    email   VARCHAR(100),
    age     INT
);

DESC students;
```

```
13
14  -- Optional test rows (you can also add them from addStudent.jsp)
15  -- INSERT INTO students (roll_no, name, email, age) VALUES
16  -- (101, 'Rahul', 'rahul@gmail.com', 20),
17  -- (102, 'Priya', 'priya@gmail.com', 19);
```

DBConnection.java JAVA

```java
1  package com.studentcrud;
2
3  import java.sql.Connection;
4  import java.sql.DriverManager;
5  import java.sql.SQLException;
6
7  public class DBConnection {
8      private static String jdbcURL = "jdbc:mysql://localhost:3306/studentDB";
9      private static String jdbcUsername = "root";
10     private static String jdbcPassword = "root123";
11
12     public static Connection getConnection() {
13         Connection connection = null;
14         try {
15             Class.forName("com.mysql.cj.jdbc.Driver");
16             connection = DriverManager.getConnection(jdbcURL, jdbcUsername, jdbcPassword);
17         } catch (ClassNotFoundException | SQLException e) {
18             e.printStackTrace();
19         }
20         return connection;
21     }
22 }
```

Student.java JAVA

```java
1  package com.studentcrud;
2
3  public class Student {
4      private int rollNo;
5      private String name;
6      private String email;
7      private int age;
8
9      // Getters
10     public int getRollNo() {
11         return rollNo;
12     }
13
14     public String getName() {
15         return name;
16     }
17
18     public String getEmail() {
19         return email;
20     }
21
22     public int getAge() {
23         return age;
24     }
25
26     // Setters
27     public void setRollNo(int rollNo) {
```

```
        this.rollNo = rollNo;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public void setAge(int age) {
        this.age = age;
    }

    // toString method
    @Override
    public String toString() {
        return "Student{" +
                "rollNo=" + rollNo +
                ", name='" + name + '\'' +
                ", email='" + email + '\'' +
                ", age=" + age +
                '}';
    }
}
```

StudentDAO.java JAVA

```
package com.studentcrud;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class StudentDAO {

    public void addStudent(Student student) {
        String sql = "INSERT INTO students (roll_no, name, email, age) VALUES (?, ?, ?, ?)";
        try (Connection connection = DBConnection.getConnection();
             PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
            preparedStatement.setInt(1, student.getRollNo());
            preparedStatement.setString(2, student.getName());
            preparedStatement.setString(3, student.getEmail());
            preparedStatement.setInt(4, student.getAge());
            preparedStatement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public List<Student> getAllStudents() {
        List<Student> students = new ArrayList<>();
        String sql = "SELECT * FROM students";
        try (Connection connection = DBConnection.getConnection();
             Statement statement = connection.createStatement();
             ResultSet resultSet = statement.executeQuery(sql)) {
            while (resultSet.next()) {
                Student student = new Student();
                student.setRollNo(resultSet.getInt("roll_no"));
                student.setName(resultSet.getString("name"));
                student.setEmail(resultSet.getString("email"));
```

```
                student.setAge(resultSet.getInt("age"));
                students.add(student);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return students;
    }

    public void updateStudent(Student student) {
        String sql = "UPDATE students SET name = ?, email = ?, age = ? WHERE roll_no = ?";
        try (Connection connection = DBConnection.getConnection();
             PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
            preparedStatement.setString(1, student.getName());
            preparedStatement.setString(2, student.getEmail());
            preparedStatement.setInt(3, student.getAge());
            preparedStatement.setInt(4, student.getRollNo());
            preparedStatement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public void deleteStudent(int rollNo) {
        String sql = "DELETE FROM students WHERE roll_no = ?";
        try (Connection connection = DBConnection.getConnection();
             PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
            preparedStatement.setInt(1, rollNo);
            preparedStatement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

NOTE `com.mysql.cj.jdbc.Driver` is the driver class of MySQL Connector/J 8.x (the old `com.mysql.jdbc.Driver` is deprecated); `root123` is the MySQL root password on the lab PC.

addStudent.jsp JSP

```
<%@ page import="com.studentcrud.Student, com.studentcrud.StudentDAO" %>
<%
    if(request.getParameter("rollNo") != null) {
        int rollNo = Integer.parseInt(request.getParameter("rollNo"));
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        int age = Integer.parseInt(request.getParameter("age"));
        Student student = new Student();
        student.setRollNo(rollNo);
        student.setName(name);
        student.setEmail(email);
        student.setAge(age);
        StudentDAO dao = new StudentDAO();
        dao.addStudent(student);
        out.println("Student added successfully.");
    }
%>
<html>
<body>
<h2>Add Student</h2>
```

```
<form action="addStudent.jsp" method="post">
    Roll No: <input type="text" name="rollNo"><br>
    Name: <input type="text" name="name"><br>
    Email: <input type="text" name="email"><br>
    Age: <input type="text" name="age"><br>
    <input type="submit" value="Add Student">
</form>
<a href="viewStudents.jsp">View Students</a>
</body>
</html>
```

viewStudents.jsp JSP

```
<%@ page import="java.util.List, com.studentcrud.Student, com.studentcrud.StudentDAO" %>
<%
    StudentDAO dao = new StudentDAO();
    List<Student> students = dao.getAllStudents();
%>
<html>
<body>
<h2>View Students</h2>
<table border="1">
    <tr>
        <th>Roll No</th>
        <th>Name</th>
        <th>Email</th>
        <th>Age</th>
        <th>Actions</th>
    </tr>
    <% for(Student student : students) { %>
    <tr>
        <td><%= student.getRollNo() %></td>
        <td><%= student.getName() %></td>
        <td><%= student.getEmail() %></td>
        <td><%= student.getAge() %></td>
        <td>
            <a href="updateStudent.jsp?rollNo=<%= student.getRollNo() %>">Edit</a> |
            <a href="deleteStudent.jsp?rollNo=<%= student.getRollNo() %>">Delete</a>
        </td>
    </tr>
    <% } %>
</table>
<a href="addStudent.jsp">Add Student</a>
</body>
</html>
```

updateStudent.jsp JSP

```
<%@ page import="com.studentcrud.Student, com.studentcrud.StudentDAO" %>
<%
    StudentDAO dao = new StudentDAO();
    int rollNo = Integer.parseInt(request.getParameter("rollNo"));
    Student student = new Student();
    for(Student s : dao.getAllStudents()) {
        if(s.getRollNo() == rollNo) {
            student = s;
            break;
        }
    }
    if(request.getParameter("name") != null) {
```

```
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        int age = Integer.parseInt(request.getParameter("age"));
        student.setName(name);
        student.setEmail(email);
        student.setAge(age);
        dao.updateStudent(student);
        out.println("Student updated successfully.");
    }
%>
<html>
<body>
<h2>Update Student</h2>
<form action="updateStudent.jsp?rollNo=<%= student.getRollNo() %>" method="post">
    Name: <input type="text" name="name" value="<%= student.getName() %>"><br>
    Email: <input type="text" name="email" value="<%= student.getEmail() %>"><br>
    Age: <input type="text" name="age" value="<%= student.getAge() %>"><br>
    <input type="submit" value="Update Student">
</form>
<a href="viewStudents.jsp">View Students</a>
</body>
</html>
```

deleteStudent.jsp JSP

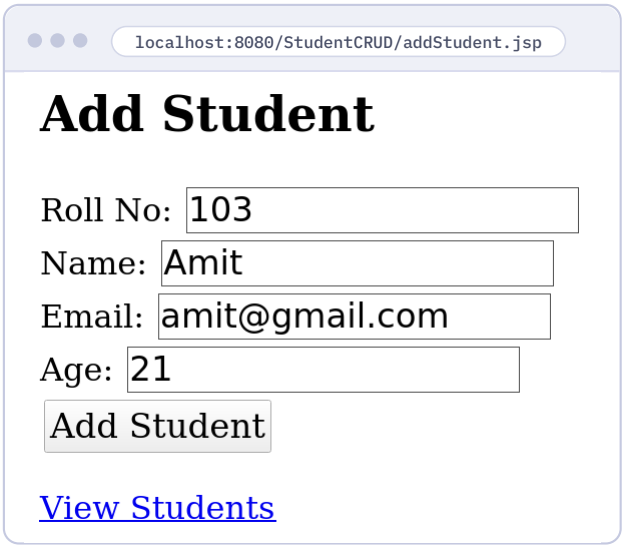
```
<%@ page import="com.studentcrud.StudentDAO" %>
<%
    int rollNo = Integer.parseInt(request.getParameter("rollNo"));
    StudentDAO dao = new StudentDAO();
    dao.deleteStudent(rollNo);
    out.println("Student deleted successfully.");
%>
<html>
<body>
<a href="viewStudents.jsp">View Students</a>
</body>
</html>
```

OUTPUT

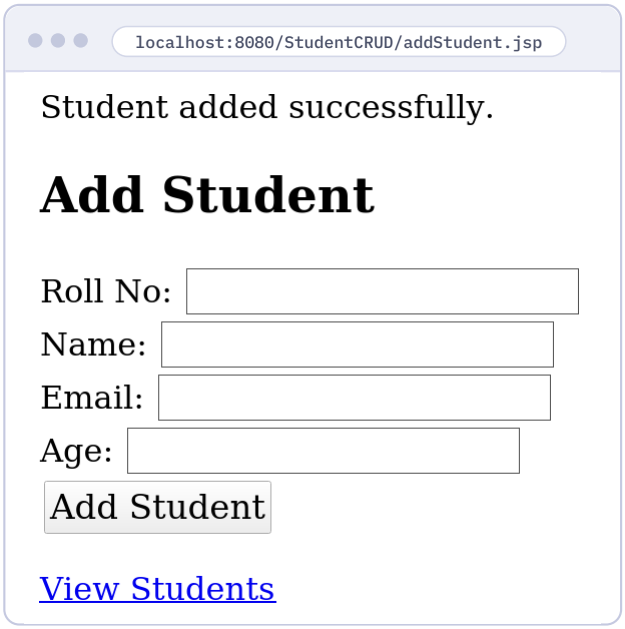
MYSQL

```
mysql> DESC students;
+---------+-------------+------+-----+---------+-------+
| Field   | Type        | Null | Key | Default | Extra |
+---------+-------------+------+-----+---------+-------+
| roll_no | int         | NO   | PRI | NULL    |       |
| name    | varchar(50) | YES  |     | NULL    |       |
| email   | varchar(100)| YES  |     | NULL    |       |
| age     | int         | YES  |     | NULL    |       |
+---------+-------------+------+-----+---------+-------+
```

- **Create – a new student is added with addStudent.jsp**

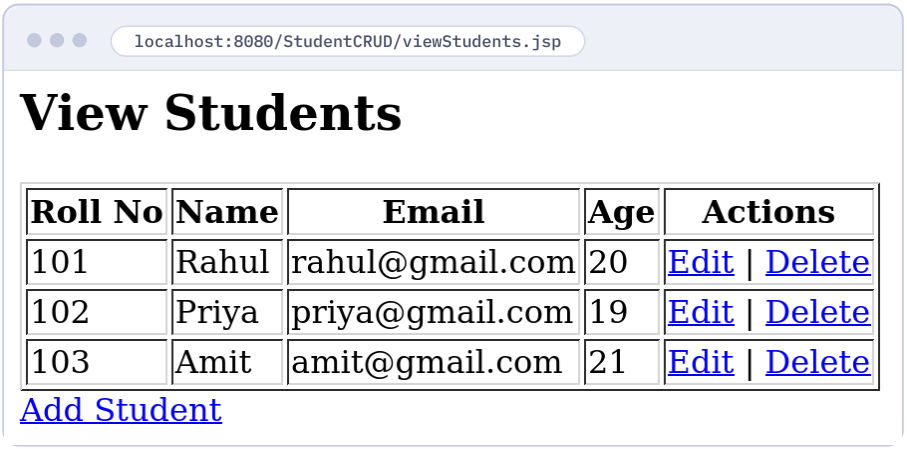


addStudent.jsp with the form filled in



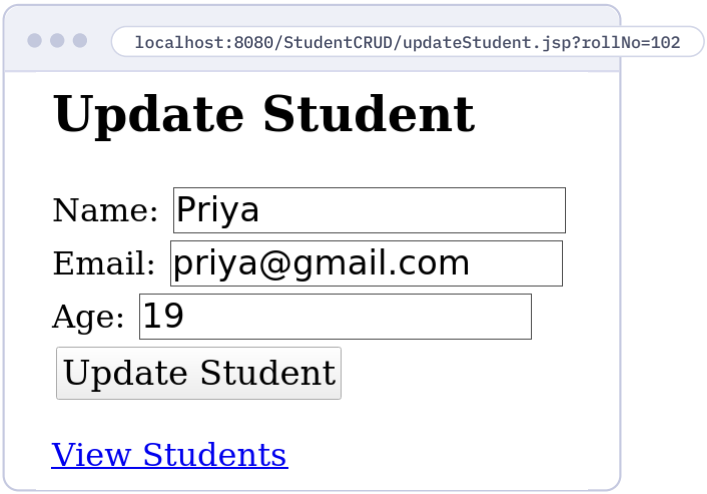
After clicking Add Student

- **Read – all records are listed by viewStudents.jsp**

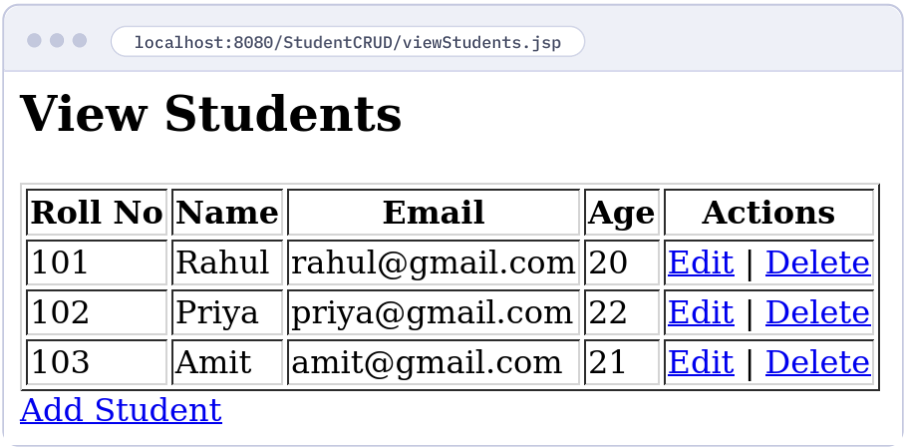


Records stored in the students table

- **Update – Edit is clicked for roll number 102 and the age is changed from 19 to 22**

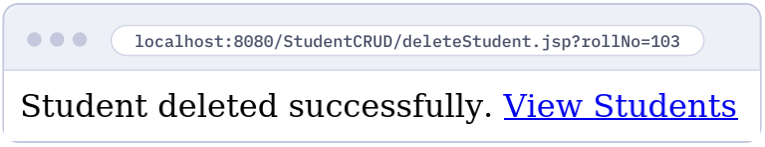


updateStudent.jsp with the existing values

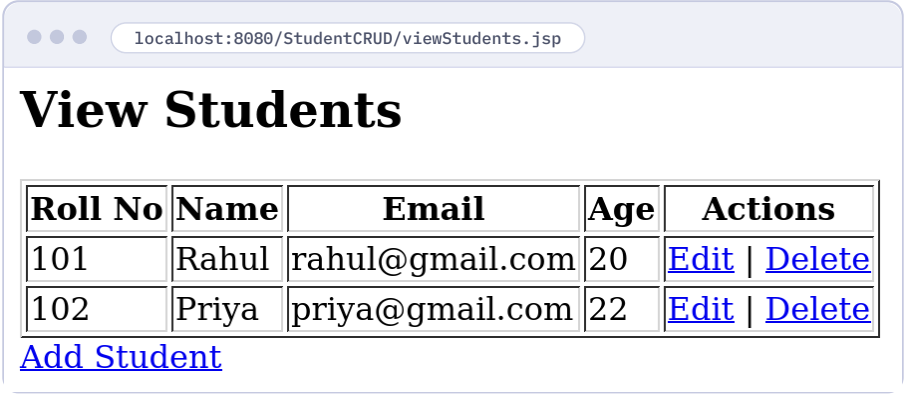


viewStudents.jsp after the update

- **Delete – Delete is clicked for roll number 103**



Message shown by deleteStudent.jsp



viewStudents.jsp after the deletion

- **Verification in MySQL**

MYSQL

```
mysql> SELECT * FROM students;
+---------+-------+----------------+------+
| roll_no | name  | email          | age  |
+---------+-------+----------------+------+
|     101 | Rahul | rahul@gmail.com |   20 |
|     102 | Priya | priya@gmail.com |   22 |
+---------+-------+----------------+------+
```

Appendix A

Practical Execution & Compatibility Guide · CS3603-1 · Prajwal Jogi

A practical companion to the experiments in this manual: which Java and Tomcat versions to use, where every file goes in an Eclipse Dynamic Web Project, how to run each experiment, and the Eclipse shortcuts worth knowing for the lab exam.

SEC. NO.	TITLE	PAGE NO.
1	The Truth About Eclipse & Java Versions Why Tomcat 9 (not 10+) matters, and which JDK to use	28
2	Eclipse Web Project Folder Hierarchy Which file goes in which folder of a Dynamic Web Project	28
3	Experiment-by-Experiment Execution Guide Setup, run steps and URLs for Expts. 1A, 1B, 2A, 2B and 3	29
4	Mastering Imports & Eclipse Exam Shortcuts Organize Imports, wildcard-import pitfalls, exam-day shortcuts	33

Java and Spring Framework Laboratory

Course Code: CS3603-1 — NMAM Institute of Technology

Comprehensive Lab Exam Execution, Folder Architecture & Version Guide

1 The Truth About Eclipse & Java Versions

A common source of confusion is whether code written in newer versions of Eclipse (such as 2024, 2025, or 2026) will run properly on college lab machines.

Definitive Takeaway on Versions

Eclipse itself is simply a code editor and project builder — it NEVER breaks your code. Whether your college PC runs Eclipse 2019, 2020, 2024, or 2026, standard Java code executes identically. What actually runs your code is **Java (JDK)** and the **Tomcat Web Server**.

The Only Real Incompatibility Trap: Tomcat 9 vs. Tomcat 10+

- **Tomcat 9.0.x (Correct):** Uses the classic Java EE namespace: `import javax.servlet.*`. All code in your NMAMIT lab manual is written for Tomcat 9.
- **Tomcat 10.x / 11.x (Incompatible with Manual Code):** Switched to Jakarta EE: `import jakarta.servlet.*`. If you run the manual code on Tomcat 10+, it fails with `ClassNotFoundException`.
- **Conclusion:** Always select **Apache Tomcat v9.0** as your Target Runtime in Eclipse.

Java Versions (Java 11, 17, 21)

Your lab manual requires OpenJDK 11 or later. Your Linux VM runs **OpenJDK 17 LTS**. All Servlets, JSPs, and JDBC code compile and execute with 100% compatibility across Java 11, 17, and 21.

2 Eclipse Web Project Folder Hierarchy

In Eclipse, a **Dynamic Web Project** has a strict directory structure. Putting a file in the wrong folder is the number one reason students encounter 404 Not Found or ClassNotFound errors.

Folder in Eclipse	What Goes Inside It
`src/main/java`	**All Java source files (.java)**: Servlets, DAO classes, POJOs, and DB helper classes. Must reside inside appropriate packages (e.g. `com.example`).
`src/main/webapp/`	**All web documents (.html, .jsp)**: Forms, result pages, CSS, images. These are directly accessible by web browsers via URL.
`src/main/webapp/WEB-INF/lib/`	**External JAR libraries**: `mysql-connector-j-8.x.jar`. Files placed here are automatically included in Tomcat's runtime classpath.
`src/main/webapp/WEB-INF/`	Contains `web.xml`. Never place regular JSP or HTML files directly here unless you intentionally want to block browser URL access.

3 Experiment-by-Experiment Execution Guide

Experiment 1A: Servlet Handling HTTP GET and POST Requests

- **Project Creation:** File → New → Dynamic Web Project → Name: `ServletGetPostDemo`. Set **Target runtime**: `Apache Tomcat v9.0`, **Dynamic web module version**: `4.0`.
- **Java File Location:** Under `src/main/java`, create package `com.example`, then create class `SimpleServlet.java`.
- **Imports Used:**

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
```

- **Execution:** Right-click project → **Run As** → **Run on Server** → Finish.
- **Browser URL:** http://localhost:8080/ServletGetPostDemo/SimpleServlet
- **Behavior:** Initial GET loads HTML form with Name & Password inputs. Clicking Submit sends a POST request that echoes the submitted credentials.

Experiment 1B: Session Tracking Using Cookies & Session Objects

- **Project Name:** `SessionServletDemo` (or inside the same project).
- **Java File Location:** `src/main/java/com/example/SessionServlet.java`.
- **Key Imports:** Includes `javax.servlet.http.HttpSession`. Notice: Qualify `new java.util.Date(session.getCreationTime())` to avoid collision with SQL Date.
- **Login Credentials:** Username: `admin`, Password: `password`.
- **Browser URL:** http://localhost:8080/SessionServletDemo/SessionServlet
- **Behavior:** Displays login form. Upon login, stores user in `HttpSession`, prints Session ID, Creation Time, and Last Accessed Time. Provides a working Logout link that calls `session.invalidate()`.

Experiment 2A: JSP Application for Employee Information

- **Project Name:** `JSPEmpInfo` (Dynamic Web Project, Tomcat 9.0, Module 4.0).
- **File Locations (Both in `src/main/webapp/`):**
  1. `src/main/webapp/employeeForm.jsp`: Contains HTML form collecting Employee ID, Name, Age, Department, Email with `method="POST"` pointing to `employeeResult.jsp`.
  2. `src/main/webapp/employeeResult.jsp`: Contains `<%@ page import="java.util.Calendar" %>`, retrieves values using `request.getParameter()`, computes birth year (`Calendar.YEAR - age`), and prints a formatted HTML table.
- **Execution:** Right-click `employeeForm.jsp` → Run As → Run on Server.
- **Browser URL:** http://localhost:8080/JSPEmpInfo/employeeForm.jsp

Experiment 2B: Server-Side Validation Using JSP

- **Project Name:** `JSPEmpValidate`.
- **File Locations (Both in `src/main/webapp/`):**
  1. `employeeForm.jsp`: Form with department dropdown (`HR, Finance, IT, Sales`).
  2. `employeeResult.jsp`: Validates:
     - Name contains only letters and spaces (`^[a-zA-Z\\s]+$`).
     - Age between 18 and 65 (`Integer.parseInt(ageStr)`).
     - Department matches allowlist (`HR|Finance|IT|Sales`).
     - Email matches format (`^[\\w.-]+@[\\w.-]+\\.[a-zA-Z]{2,}$`).

     If invalid, displays specific bullet errors and a "Go back" link.
- **Browser URL:** http://localhost:8080/JSPEmpValidate/employeeForm.jsp

Experiment 3: JSP and JDBC CRUD Operations on Students Table

This is the most comprehensive lab experiment. Follow these four exact stages:

Stage 1: MySQL Database Initialization

In Ubuntu terminal or MySQL CLI:

```
CREATE DATABASE IF NOT EXISTS studentDB;
USE studentDB;
CREATE TABLE IF NOT EXISTS students (
    roll_no INT PRIMARY KEY,
    name VARCHAR(50),
    email VARCHAR(100),
    age INT
);
```

Stage 2: Project Setup & Connector JAR

1. Create Dynamic Web Project: `StudentCRUD` (Tomcat 9.0, Dynamic Web Module 4.0).
2. Copy `mysql-connector-java-8.0.26.jar` into `src/main/webapp/WEB-INF/lib/`.
3. Right-click project → **Refresh** (F5), then **Project** → **Clean...**

Stage 3: Java Classes under `src/main/java/com/studentcrud/`

- `DBConnection.java`:

```
package com.studentcrud;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    private static String jdbcURL = "jdbc:mysql://localhost:3306/
       studentDB";
    private static String jdbcUsername = "root";
    private static String jdbcPassword = "root123";

    public static Connection getConnection() {
        Connection connection = null;
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            connection = DriverManager.getConnection(jdbcURL,
               jdbcUsername, jdbcPassword);
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
        }
        return connection;
    }
}
```

- `Student.java`: POJO with attributes (`rollNo`, `name`, `email`, `age`), getters, and setters.
- `StudentDAO.java`: Implements `addStudent()`, `getAllStudents()`, `updateStudent()`, and `deleteStudent()` using JDBC `PreparedStatement`.

Stage 4: JSP Views under `src/main/webapp/`

- `viewStudents.jsp`: Displays all student records in a table with Edit and Delete links.
- `addStudent.jsp`: Form that inserts a new student record into `studentDB`.
- `updateStudent.jsp`: Pre-populates selected student details and executes SQL UPDATE.
- `deleteStudent.jsp`: Deletes record by `rollNo` parameter and shows confirmation.

Execution & URL

Add project to Tomcat server. Open in browser:
`http://localhost:8080/StudentCRUD/viewStudents.jsp`

4 Mastering Imports & Eclipse Exam Shortcuts

Critical Warning: Letter 'O' vs. Number '0' (Zero)

- **Ctrl + Shift + O (Capital Letter O, between I and P):**
  → Stands for **Organize Imports**. This is the true shortcut that automatically inserts all missing imports and cleans up your file.
- **Ctrl + Shift + 0 (Number Zero on top row):**
  → **DOES NOT WORK!** In Eclipse, the number 0 has no organize import function. In Windows and Linux, it either switches keyboard input layouts or does absolutely nothing.
- **100% Fail-Safe Mouse Method (Never Fails):**
  If you are ever in doubt, **Right-click** inside the editor → select **Source** → click **Organize Imports**.

Practical Test: Writing Code Without Any Imports

Suppose you write the following method inside `StudentDAO.java` without typing a single `import` statement:

```
package com.studentcrud;

public class StudentDAO {
    public void getStudentList() {
        List<Student> list = new ArrayList<>();
        Connection conn = null;
        PreparedStatement ps = null;
    }
}
```

1. Before Pressing the Shortcut (Errors Visible)

Because `List`, `ArrayList`, `Connection`, and `PreparedStatement` are not yet imported, Eclipse places **red squiggly error lines** under all four:

- `List cannot be resolved to a type`
- `ArrayList cannot be resolved to a type`
- `Connection cannot be resolved to a type`
- `PreparedStatement cannot be resolved to a type`

2. If You Press Ctrl + Shift + 0 (Digit Zero)

Nothing happens. The red squiggly lines remain because zero is the wrong key.

3. When You Press Ctrl + Shift + O (Letter O)

All red lines vanish immediately! Eclipse inspects its classpath and automatically adds these four imports at the very top:

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.util.ArrayList;
import java.util.List;
```

Why Did `import java.util.*;` Get Erased When Pressing Ctrl + Shift + O?

Ctrl + Shift + O does two simultaneous actions:

1. Scans the code you actually wrote and automatically inserts missing imports at the top.
2. **Deletes (erases) any import that is currently unused.**

If you type `import java.util.*;` before you have written any code using classes from `java.util` (such as `List` or `Calendar`), Eclipse considers it redundant and removes it to keep the file clean.

Why Wildcard (*) Imports Cause Exam Errors

Your professor correctly warned against writing `import ...*`. Here is the technical reason:

1. **Date Conflict:** If you write `import java.util.*;` and `import java.sql.*;` together, typing `Date d;` causes a compilation error: *"The type Date is ambiguous"*.
2. **Sub-packages Are Not Imported:** Writing `import javax.servlet.*;` does NOT import `HttpServlet` or `HttpSession`. Those reside in the sub-package `javax.servlet.http.*`.
3. **JSP Validator Bug:** Writing `<%@ page import="com.studentcrud.*" %>` in JSP files often triggers false red error lines in Eclipse's editor. Always explicitly import exact classes: `<%@ page import="com.studentcrud.Student, com.studentcrud.StudentDAO" %>`.

Crucial Rule: Java Files vs. JSP Files

- **Java Files (.java):** `Ctrl + Shift + O` works 100% of the time.
- **JSP Files (.jsp):** `Ctrl + Shift + O` **does NOT work** inside JSP files. You must write `<%@ page import="..." %>` manually at line 1.

Essential Exam Day Shortcuts (Guaranteed Available in All Versions)

- **Ctrl + Shift + O (Letter O)**: Auto-import required packages / clean unused ones.
- **Right-click → Source → Organize Imports**: Mouse alternative to shortcut.
- **Ctrl + Shift + F**: Auto-format and perfectly align messy indentation and braces.
- **Ctrl + Space**: Context-aware code completion (methods, variables, boilerplate).
- **sysout + Ctrl + Space**: Expands instantly to `System.out.println();`.
- **Ctrl + S**: Save and trigger automatic Eclipse background compilation.